

1) Evaluation Criteria

Error image:

$$\mathcal{E}(f, \tilde{f}) = f - \tilde{f}$$

Mean Square Error (MSE):

$$\mathcal{D}(f, \tilde{f}) = \frac{1}{NM} \|\mathcal{E}\|^2$$

Peaks Signal to Noise Ratio (PSNR)

$$PSNR(f, \tilde{f}) = 10 \log_{10} \frac{255^2}{\mathcal{D}(f, \tilde{f})}$$

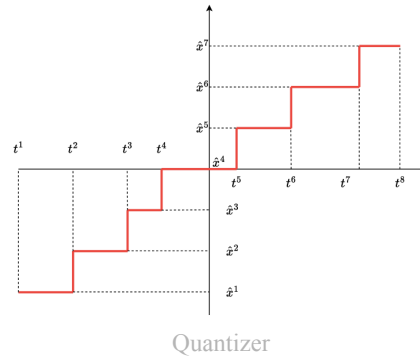
2) Quantization

The quantization is the process of mapping a function Q from $\mathbb{R}$ to a discrete set called *Dictionary*.

$$Q : x \in \mathbb{R} \rightarrow y \in C = \{\hat{x}_1, \hat{x}_2, \dots\} \subset \mathbb{R}$$

with:

- C : dictionary (subset of $\mathbb{R}$)
- $\hat{x}_i$: quantization level, codeword

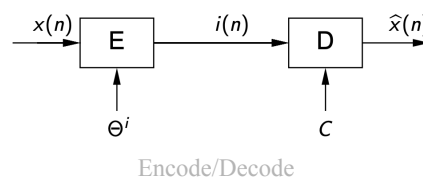


Moreover we define:

- Thresholds (t_1, t_n)
- Levels: $L = n - 1$
- Region Θ_i : Two subsequent thresholds define a region $\Theta_i = (t_i, t_{i+1}) = \{x : Q(x) = \hat{x}_i\}$ these are a partition (non intersecting intervals)
- Quantization error: $e = x - Q(x)$. The smaller the regions, the better the error.

Quantization can be seen as an encoding/decoding process:

take signal $x(n)$: the encoding step assigns each sample of $x(n)$ to a quantization level $i(n)$. The decoder associates to each quantization level $i(n)$ the corresponding code word (value).



We will refer to these steps also as quantization and inverse quantization.

Rate R :

The rate is the avg amount of bits used to store the quantization indexes $i(n)$. This is saved via lossless coding:

$$R = \log_2 L$$

- quantized data has uniform (pessimistic) distribution (since $H \leq R \iff$ signal uniform)
- binary data of $i(n)$ is the best (entropy coded)

Distortion D :

On a single sample we use

$$d[x, Q(x)] = \|e\|^2 = \|x - Q(x)\|^2$$

For a signal of duration N we use the MSE:

$$D = \frac{1}{N} \sum_{n=0}^{N-1} d[x(n), Q(x(n))]$$

for a random signal this can be done via an expected value:

$$D = \mathbb{E} [\|X(n) - Q(X(n))\|^2] = \mathbb{E} [\|E(n)\|^2] = \sigma_E^2$$

Mid treat quantizers are quantizers where values near 0 are have quantization level = 0. This reduces noise. **Mid rise** quantizers amplify noise!

2.1) Uniform Quantizer (UQ)

Input signal $\in (0, A)$ (unsigned) or $\in (-\frac{A}{2}, \frac{A}{2})$ (signed) is divided into L equal sized cells of size $\Delta = A/L$. Where the quantization levels $\hat{x}_i$ is the midpoint. clearly this is now **mid rise**

$$\begin{aligned} \forall i, \Delta^i &= \Delta = A/L \\ t^i &= t^{i-1} + \Delta \\ \hat{x}^i &= \frac{t^i + t^{i-1}}{2} \\ \Theta^i &= \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2} \right) \end{aligned}$$

2.1.1) Types of UQ

Unsigned Data

For unsigned data $\in (0, A)$ we have

Thresholds: $t_i = (i - 1)\Delta$

Encoder: $i = \left\lceil \frac{x}{\Delta} \right\rceil$

Decoder: $\hat{x}^i = i\Delta - \frac{\Delta}{2}$

$$\Rightarrow Q(x) = \Delta \left\lceil \frac{x}{\Delta} \right\rceil - \frac{\Delta}{2}$$

Signed Data

Since values are unsigned a mid thread quantizer is necessary, therefore we need an **odd number of levels**. The threshold at $t^{N/2}$ will have $\hat{x}^{N/2} = 0$.

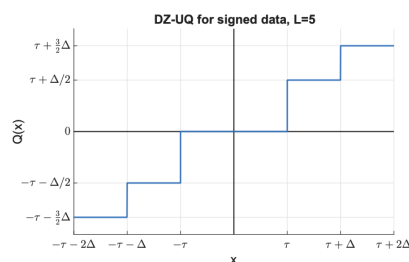
Encoder: $i = \text{round}\left(\frac{x}{\Delta}\right)$

Decoder: $\hat{x}^i = i\Delta$

$$\Rightarrow Q(x) = \Delta \text{round}\left(\frac{x}{\Delta}\right)$$

From here we can define **deadzone quantization**:

The central cell is larger than the others, it is zero if $\|x\| < \tau$



Deadzone UQ

$$i = \begin{cases} \text{sign}(x) \left\lfloor \frac{\|x\| + \frac{\tau\Delta}{2}}{\Delta} \right\rfloor & \|x\| \geq \tau \\ 0 & \|x\| < \tau \end{cases}$$

High Resolution (HR) UQ

For HR we mean a quantizer with $L \rightarrow \infty$. Therefore we get $\Delta = A/L \rightarrow 0$ and therefore Θ_i as $\Theta_i = (\hat{x}_i - \Delta/2, \hat{x}_i + \Delta/2) \rightarrow (\hat{x}_i, \hat{x}_i)$. Now we can approximate p_X as a constant in each Θ_i .

Therefore in a single region:

$$E|X \in \Theta_i = X - Q(X) = X - \hat{x} \sim \mathcal{U}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$$

Due to total probability law we have:

$$E \sim \mathcal{U}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$$

2.1.2) RD Curve

PROBABILITY THEORY RECAP

First recall the definition of uniform RV:

$$X \sim \mathcal{U}(a, b) \iff f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases} \quad \begin{aligned} \mathbb{E}[X] &= \frac{a+b}{2} \\ \text{var}(X) &= \frac{(b-a)^2}{12} \end{aligned}$$

and also the Law of the Unconscious Statistician (LOTUS):

$$\mathbb{E}[g(x)] = \int g(x) f_X(x) dx$$

Uniform RV with UQ

Hypothesis: $X \sim \mathcal{U}(-\frac{A}{2}, \frac{A}{2})$ quantized with UQ of L levels.

Goal: Since the rate is known ($R = \log_2 L$) we must only find the distortion:

$$D = \sigma_Q^2 = \mathbb{E}[\|E\|^2] = \mathbb{E}\left[\|X - \hat{X}\|^2\right].$$

Result:

$$D(R) = \sigma_Q^2 = \sigma_X^2 2^{-2R}$$

Proof:

Notice that since $\mathbb{E}\left[\|X - \hat{X}\|^2\right] = \mathbb{E}[g(X)]$ we can use LOTUS:

$$\begin{aligned} \sigma_Q^2 = \mathbb{E}[g(X)] &= \int_{-\frac{A}{2}}^{\frac{A}{2}} g(u) \frac{1}{A} du = \sum_{i=1}^L \int_{\Theta_i} \frac{1}{A} [u - Q(u)]^2 du \\ &= \frac{1}{A} \sum_{i=1}^L \int_{\hat{x}_i - \Delta/2}^{\hat{x}_i + \Delta/2} [u - \hat{x}_i]^2 du \\ &= \frac{1}{A} \sum_{i=1}^L \int_{-\Delta/2}^{\Delta/2} t^2 dt \\ &= \frac{A}{L} \frac{\Delta^3}{12} \end{aligned}$$

finally recall that in UQ we have $\Delta = A/L$, that $L = 2^R$ and that $\text{var}(X) = A^2/12$ then:

$$\sigma_Q^2 = \frac{A}{L} \frac{\Delta^3}{12} = \frac{\Delta^2}{12} = \frac{1}{12} \left(\frac{A}{L}\right)^2 = \frac{A^2}{12} \frac{1}{L^2} = \sigma_X^2 2^{-2R}$$

and the SNR becomes:

$$SNR = 10\log_{10} \frac{\mathbb{E}[X^2]}{D} = 10\log_{10} \frac{\sigma_X^2}{\sigma_X^2 2^{-2R}} = 10\log_{10} 2^{2R} \approx 6R$$

□

HRUQ RD Curve

In this case we know $E \sim \mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2})$ and thus

$$D = \mathbb{E}[\|E\|^2] = \frac{\Delta^2}{12} = \frac{1}{12} \left(\frac{A}{L} \right)^2 = \frac{A^2}{12} 2^{-2R}$$

The SNR becomes:

$$SNR = 10\log_{10} \frac{\mathbb{E}(X^2)}{D} = 10\log_{10} \frac{\sigma_X^2 2^{-2R}}{A^2/12} \approx 6R - 10\log_{10} \frac{\gamma^2}{2}$$

where γ^2 is the **load factor**, that is, the ratio between peak power and avg power:

$$\gamma^2 = \frac{X_{\max}^2}{\sigma_X^2} = \frac{A^2/4}{\sigma_X^2}$$

2.1.3) Recap:

Here is a table:

	Unsigned	Signed	Deadzone (Signed)	High Resolution
Type	Mid-rise	Mid-tread	Mid-tread (large center)	Any ($L \rightarrow \infty$)
Implementation: $Q(x)$	$\Delta \cdot \left\lfloor \frac{x}{\Delta} \right\rfloor + \frac{\Delta}{2}$	$\Delta \cdot \text{round} \left(\frac{x}{\Delta} \right)$		$\Delta \cdot \text{round} \left(\frac{x}{\Delta} \right)$
L	Even	Odd	Odd	Any
RD Curve	-	if uniform input $\sigma_X^2 2^{-2R}$		$\frac{\gamma^2}{3} \sigma_X^2 2^{-2R}$

Given range and levels, UQ has the **smalles maximum error** (optimal minimax quantizer) of $e_{\max} = \frac{\Delta_i}{2} = \frac{A}{2L}$

Only quantizer with analytical solution for RD curve with uniform input distribution (see [Uniform RV with UQ](#)):

$$D(R) = \sigma_X^2 2^{-2R}$$

in the general case (no knowledge on input distribution):

$$D(R) = \frac{A^2}{12} 2^{-2R}$$

this is used for HRUQ also. Recall γ^2 is max power over avg power and $c_X = \frac{\gamma^2}{3}$.

2.2) Optimal Scalar Quantization (SQ)

Let the input density p_X be known. We want to find the quantizer minimizing the distortion (for a given rate).

Optimal HR Quantizer

Recall the HR hypothesis:

$$L \rightarrow \infty \quad \max_i \Delta_i \rightarrow 0 \quad \forall i, u \in \Theta^i, p_X(x) \approx P_i$$

It can be shown that the optimal quantizer is:

$$\sigma_Q^2 = c_X \sigma_X^2 2^{-2R} \quad \text{with } c_X = \frac{1}{12} \left[\int p_U^{1/3}(t) dt \right]^3 \text{ and } U = \frac{X}{\sigma_X}$$

The term c_X is called **shape factor** since it only depends on the PDF shape and not variance (U and X have same shape)

some common shape factors are:

- Uniform: $c_X = 1$
- Gaussian $c_X = \frac{\sqrt{3}}{2} \pi \approx 2.72$

Non-HR Optimal Quantization (TODO)

There is **no analytical formula** for low rate optimal quantizers, but it is possible to find **necessary conditions** that allow to defining the **Lloyd-Max algorithm** to find local optimum points

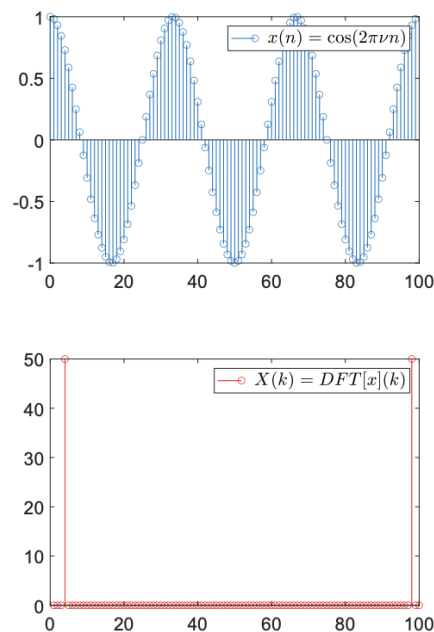
2.3) Predictive SQ

Quantization is **not effective for non sparse data**. Predictive SQ **exploits correlation among samples**.

Definition 1 (Sparse Signal).

A signal is sparse if most of its components are zero or close to zero

- The variance of a sparse signal is low
- Zero (or close to zero) samples can be neglected (quantized to 0) without introducing distortion
- Natural signal are not sparse but can easily be transformed into one



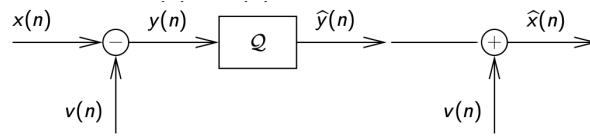
Example non sparse vs sparse representation of sinusoidal

Optimization Paradigm

Since prediction error = signal error, then performance increase depends only by predictor variance, that is **if and only if the pred error has smaller variance than og signal**

Proof prediction error=signal error:

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - (\hat{x}(n) - v(n)) = x(n) - \hat{x}(n) = \bar{q}(n)$$



Quantizer process

Proof SNR quality depends on σ_Y^2

$$SNR = 10\log_{10} \frac{\sigma_X^2}{D} = 10\log_{10} \frac{\sigma_X^2}{\sigma_Y^2} + 10\log_{10} \frac{\sigma_Y^2}{D} = G_P + G_Q$$

Predictors

Linear predictors are used: simple and optimal for gaussian rvs.

A linear predictor is a linear combination of the P previous values

$$v(n) = - \sum_{i=1}^P a_i x_{n-i} \rightarrow y(n) = x(n) - v(n) = \sum_{i=0}^P a_i x_{n-i}, \quad a_0 = 1$$

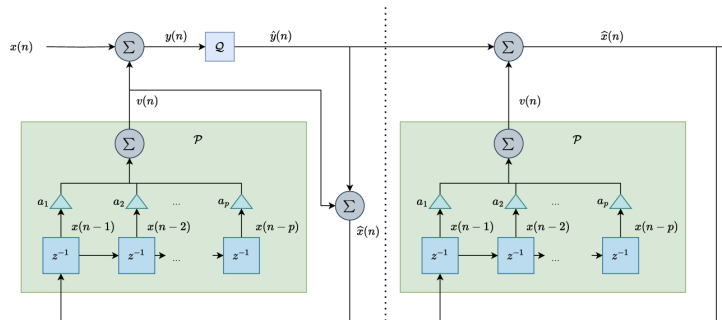
It turns out that the optimal variance is:

$$\sigma_Y^2 = \sigma_X^2 + r^T a^* = \sigma_X^2 - r^T R_X^{-1} r$$

where $a^* = -R_X^{-1} r$

The order of the filter yields good result for small values (up to order 4), but including distant pixels (less correlation) leads to more white noise inclusion and has higher complexity for negligible PSNR increases.

Clearly now v_n must use $\hat{x}$ since otherwise the decoder wouldn't be able to use know v .



Block Scheme

The use of $\hat{x}$ instead of x comes with some heavy performance penalty, it performs worse than direct quantization. This can be fixed via entropy coding.

Finally, a **local adaptation with block wise approach** can be used. Divide image into $M \times M$ blocks and find the optimal filter for each block. The rate augments by $\frac{NB}{M^2}$:

- Small block: good local statistics, bad quantizer info overhead (high rate)
- Large block: bad local statistics (bad PSNR), good quantizer overhead

Let $X(n) \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{|n-m|}$ and $V(n) = X(n-1)$. Find ρ such that $G_p > 0$.

Since we need $G_p > 0$ we actually need $\sigma_Y^2 > \sigma^2$.
Find Y

$$Y(n) = X(n) - V(n) = X(n) - X(n-1)$$

since this is a sum of gaussians this remains a zero mean gaussian rv, find variance:

$$\begin{aligned}\sigma_Y^2 &= \mathbb{E}[(X(n) - X(n-1))^2] = \mathbb{E}[X(n)^2] + \mathbb{E}[X(n-1)^2] - 2\mathbb{E}[X(n)X(n-1)] \\ &= 2\sigma^2 - 2\sigma^2\rho \\ &= 2\sigma^2(1 - \rho)\end{aligned}$$

$$\sigma_Y^2 = 2\sigma^2(1 - \rho) > \sigma^2 \iff \rho < \frac{1}{2} \iff G_p > 0$$

Proof of optimal filter:

Notice that the predicted output is

$$y(n) = (a * x)(n) \xrightarrow{Z} Y(z) = A(z)X(z)$$

where the transfer function is

$$A(z) = \sum_{i=0}^P a_i z^{-i} = 1 + a_1 z^{-1} + \dots + a_P z^{-P}$$

It is clear that the minimization problem only acts on finding a that minimizes the variance.

In general the variance can be written as:

$$\sigma_Y^2 = \sigma_X^2 + 2r^T a + a^T R_X a$$

where:

- r is the autocorrelation vector
- R_X the autocorrelation matrix

The minimization can be computed:

$$\frac{\partial \sigma_Y^2}{\partial a} = 2r + 2R_X a = 0 \rightarrow a^* = -R_X^{-1}r$$

Then the optimal variance becomes:

$$\sigma_Y^2 = \sigma_X^2 + r^T a^* = \sigma_X^2 - r^T R_X^{-1} r$$

3) Lossless Coding

Lossless coding means to decrease the number of bits needed to encode the data without losing any information: the process is reversible.

This is used in the quantization part: remap the indices $i(n)$ in order to optimize bitstream.

We must understand:

- **Theoretical Bound:** Source entropy
- **Practical Implementation:** algorithm efficiency and scalability
- **Possible without known statistics?**

Lets start with some notation:

- **Alphabet:** $\mathcal{X} = \{x_1, \dots, x_M\}$ set of symbols to encode
- **Code:** application between $\mathcal{X}$ and $\{0, 1\}$ (set of finite length bit strings)

Fixed Length Coding (FLC)

FLC assumes equiprobable alphabet: all codewords ave the same length, that is M symbols $\rightarrow L = \lceil \log M \rceil$ bits to encode each symbol (bpS) and a rate of $R = \log_2 L$

Example (FLC Text Compression).

An alphabet with 26 symbols is encoded with $\lceil \log 26 \rceil = \lceil 4.7 \rceil = 5$ bits per codeword. This has a compression ratio of 1.

Variable Length Code (VLC)

Exploit input distribution to assign less bits to more probable symbols.

Parsing problem: create code where symbol is perfectly distinguishable in the bitstream

3.2) Principles of Information Theory

It is clear that there is a preferred code. VLC has a **prefix condition** where a instantaneous (prefix) code has no codeword is a prefix of another codeword.

Theorem 2 (McMillan's Theorem).

Decodable codes do not improve performance with respect to instantaneous codes

Therefore, we can focus only on instantaneous codes

Theorem 3 (Kraft's Inequality).

There exists a instantaneous code with lengths $\{l_1, \dots, l_M\}$ iff

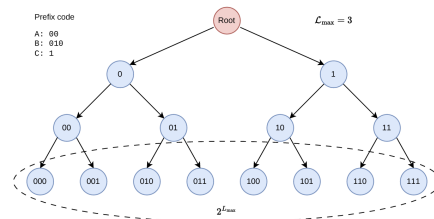
$$\sum_i 2^{-l_i} \leq 1$$

KRAFT PROOF:

Proof of Necessity ($\implies$): Codewords are already given, we must show the formula

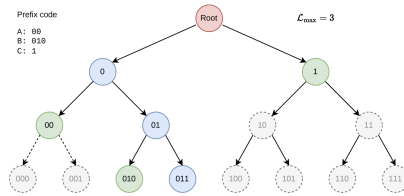
Set $L_{\max} = \max_i \{l_i\}$.

Then, by building a binary tree of the possible codes, the number of leaves (at level $L_{\max}$) is $2^{L_{\max}}$.



Binary Tree Example

Each code c_i has $2^{L_{\max}-l_i}$ leaves and due to **prefix property** no codeword is a descendant of another: leaves are disjoint. The tree can be truncated at the position of each codeword



Example Tree Truncated at Codewords

Therefore we have that the number of leaves (codewords) are necessarily less than the maximum leaves:

$$\sum_{i=1}^N 2^{L_{\max}-l_i} \leq 2^{L_{\max}}$$

$$\sum_{i=1}^N 2^{-l_i} \leq 1$$

Proof of Sufficiency ($\impliedby$): We have the formula, we must build the code

The codes can be built as follows:

0. Create a complete binary tree of depth $L_{\max}$

1. Sort all lengths $l_1 \leq \dots \leq l_N$
2. At each step $k = \{1, \dots, N\}$ pick an available position at depth l_k (increasing order)
3. Mark descendants of l_k as forbidden. $2^{L_{\max}-l_i}$ leaves are blocked.

The code can be built if at step k there is an available codeword of length l_k .

At each step the unavailable leaves (before adding c_k) are $\sum_{i=1}^{k-1} 2^{L_{\max}-l_i}$. Therefore we have $2^{L_{\max}} - \sum_{i=1}^{k-1} 2^{L_{\max}-l_i}$ available leaves.

This can be normalized wrt the number of leaves $1 - \sum_{i=1}^{k-1} 2^{-l_i}$ available leaves.

By Kraft we have

$$\begin{aligned}\sum_{i=1}^N 2^{-l_i} &= \sum_{i=1}^{k-1} 2^{-l_i} + \sum_{i=k}^N 2^{-l_i} \leq 1 \\ \sum_{i=1}^{k-1} 2^{-l_i} &\leq 1 - \sum_{i=k}^N 2^{-l_i} \\ 1 - \sum_{i=1}^{k-1} 2^{-l_i} &\geq \sum_{i=k}^N 2^{-l_i} \geq 2^{-l_k} > 0\end{aligned}$$

Since the remaining free capacity is at least 2^{-l_k} , there must exist at least one free node at depth l_k where the k -th codeword can be placed.

□

Information and Entropy Recap

Suppose symbols $\{x_i\}$ with probabilities p_i .

- The information of a symbol is

$$I(x_i) = -\log_2 p_i = \log_2 \frac{1}{p_i}$$

- Properties:

$$\begin{aligned}I(x_i) &> 0 \\ \text{if } p_i &= 1 \rightarrow I = 0 \\ \text{if } x_i \text{ indep } x_j &\rightarrow I(x_i, x_j) = I(x_i) + I(x_j)\end{aligned}$$

- The source entropy of $X = \{x_i\}$ is

$$H(X) = -\sum_i p_i \log_2 p_i = -\mathbb{E}[\log_2(p_X(x))]$$

this is the avg uncertainty of X .

- Joint Entropy:

$$H(X, Y) = -\sum_{i,j} p_{i,j} \log p_{i,j}$$

- Conditional Entropy

$$\begin{aligned}H(X|Y) &= \sum_j p_j H(X|Y = y_j) \rightarrow H(X, Y) = H(Y) + H(X|Y) \\ &= H(Y) + H(Y|X)\end{aligned}$$

- Properties:

$$\begin{aligned}H(X) &> 0 \\ H(X, Y) &= H(X) + H(Y|X) = H(Y) + H(X|Y) \\ H(X, Y) &\leq H(X) + H(Y) \text{ (= if } \perp) \\ H(X|Y) &\leq H(X) \text{ (= if } \perp) \\ H(X) &\leq \log_2 M \text{ (= if } X \sim u)\end{aligned}$$

Theorem 4 (Fundamental: Conditioning Reduces Entropy).

$$H(X|Y) \leq H(X)$$

That is: information never hurts, knowing Y we can better infer X

Lagrange's Method

Lagrange's method is a solution for minimax problems under a constraint $\phi(x) = 0$

Consider a function $f : x \in \mathbb{R}^n \rightarrow \mathbb{R}$

In order to find the maximum or minimum of f subject to the constraint $\phi(x) = 0$, we look for the stationary points of

$$J(x, \lambda) = f(x) + \lambda \phi(x), \quad \lambda \in \mathbb{R}$$

The stationary points are computed by setting to zero all the derivatives of J :

$$\frac{\partial J}{\partial x_i} = 0, \quad \frac{\partial J}{\partial \lambda} = 0$$

Example (Distribution with Maximum Entropy).

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i}, \quad \sum_i p_i = 1 \rightarrow \phi(x) = \sum_i p_i - 1 = 0$$

Write J :

$$J(p, \lambda) = - \sum_i p_i \log p_i + \lambda \left(\sum_i p_i - 1 \right)$$

Calculate the derivative (specific p_i)

$$\begin{aligned} \frac{\partial J}{\partial p_i} &= - \left(\frac{\log e}{p_i} p_i + \log p_i \right) + \lambda = 0 \rightarrow p_i = \lambda - \log e \\ \frac{\partial J}{\partial \lambda} &= \sum_{i=1}^M p_i - 1 = 0 \rightarrow p_i = 1/M \end{aligned}$$

The max uncertainty is obtained by setting all probabilities equal, that is $p_i = 1/M$.

3.3) Optimal Code

The optimal code is the solution of the following problem:

Given a set of M symbols with probabilities p_i , the optimal code is the set of lengths such that:

- $\forall i = 1, \dots, M, l_i \in \mathbb{N}$
- Kraft inequality is satisfied (prefix code)
- The average length $\mathcal{L} = \sum_{i=1}^M p_i l_i$ is minimized among all possible sets of l_i 's

The optimal solution drops the first assumption ($l_i \in \mathbb{R}$) and obtains:

$$l_i^* = -\log_2 p_i = I(x_i) \rightarrow \mathcal{L}^* = \sum_i -p_i \log_2 p_i = H(X)$$

Since real codes cannot have fractional length, this bound is achieved only if the distribution is dyadic, that is every symbol's probability is a negative power of 2:

$$\forall i \in \{1, \dots, M\}, \exists k \in \mathbb{N} \mid p_i = 2^{-k}$$

therefore in the general case $\mathcal{L}^* \geq H(X)$ but by how much?

Theorem 5 (Shannon Source Coding Theorem).

For any random source X , the minimum average length $\mathcal{L}^*$ achievable by an instantaneous (prefix) code satisfies:

$$\mathcal{L}^* \geq H(X)$$

with equality if and only if the distribution of X is **dyadic**.

PROOF:

This is a constrained optimization problem:

$$l^* = \arg \min_l \sum p_i l_i \quad \sum_i 2^{-l_i} = 1$$

$$J = \sum p_i l_i + \lambda (\sum 2^{-l_i} - 1) \rightarrow \frac{\partial J}{\partial l_i} = p_i - \lambda \cdot 2^{-l_i} \ln 2 = 0 \rightarrow p_i = \lambda \ln 2 \cdot 2^{-l_i}$$

Sum all p_i :

$$\begin{aligned} \sum_i p_i &= \lambda \ln 2 \sum 2^{-l_i} \\ 1 &= \lambda \ln 2 \end{aligned}$$

by placing this result in the initial derivative

$$\frac{\partial J}{\partial l_i} = p_i - \lambda \cdot 2^{-l_i} \ln 2 = p_i - 2^{-l_i} = 0 \rightarrow p_i = 2^{-l_i} \rightarrow l_i^* = -\log_2 p_i$$

And the avg length is then just the definition of entropy

□

The equality for Kraft was used because we represent a **complete code**. One where no possible empty branch is left

Entropy Code

To keep the first assumption we have **Entropy Coding**

$$l_i = \lceil -\log_2 p_i \rceil \rightarrow H(X) \leq \mathcal{L}^* < H(X) + 1$$

Proof:

The choice of the ceil is set a priori, therefore we must first check if Kraft holds:

$$\begin{aligned} l_i &= -\log_2 p_i + \delta_i & 0 \leq \delta < 1 \\ 2^{-l_i} &= p_i \cdot 2^{-\delta_i} = \epsilon_i p_i & \frac{1}{2} < \epsilon_i \leq 1 \end{aligned}$$

$$\sum_i 2^{-l_i} = \sum_i \epsilon_i p_i \leq \sum_i p_i = 1$$

Now the lower bound on the avg length comes directly from Shannon, while the upper bound is easily verifiable:

$$\begin{aligned} l_i &= \lceil -\log_2 p_i \rceil < -\log_2 p_i + 1 \\ p_i l_i &< p_i - p_i \log_2 p_i \\ \mathcal{L} &= \sum p_i l_i < \sum p_i - \sum p_i \log_2 p_i = 1 + H(X) \end{aligned}$$

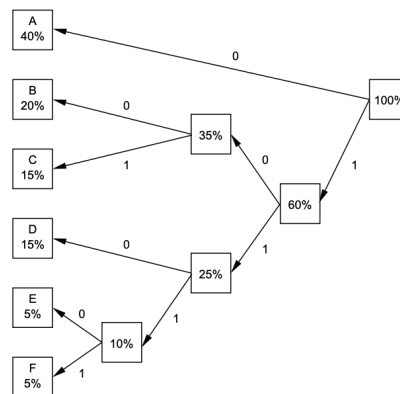
Huffman Code

Huffman discovered how to build the **optimal lossless coder for any source with known probabilities**.

Algorithm:

1. Create leaf nodes for each symbol, weighted by p_i

2. While there is more than one node:
 - Select two nodes with lowest weights
 - Create a new internal node as their parent
 - Set new node's weight as sum of children's weights
3. Assign '0' to left (upper) edges, '1' to right (lower) edges
4. Read code for each symbol from root to leaf



Example

This code has avg length $L^* = 2.3$ and entropy $H(X) = 2.246$

However now consider the following example:

Let there be a BW text image. Clearly there are many more white pixels, therefore:

$$P(X = B) = p \ll 1 \quad P(X = W) = 1 - p$$

The entropy now becomes

$$H(X) \ll 1$$

However the avg length is exactly 1.

Here the entropy is very small but the coding has high rate. This is due to a too small alphabet. It is possible to expand the alphabet by considering blocks of K symbols:

Block Coding

Instead of mapping one symbol into one codeword, we can map many subsequent symbols into a single codeword.

$$X^K = X_1 X_2 \dots X_K$$

The entropy is

$$H(X^K) \leq \sum_{i=1}^K H(X_i)$$

Theorem 6 (Entropic Rate).

We define the Entropy Rate as

$$\mathcal{H}(X) = \lim_{K \rightarrow \infty} \frac{H(X^K)}{K}$$

For stationary processes it can be shown that

$$\mathcal{H}(X) \leq H(X)$$

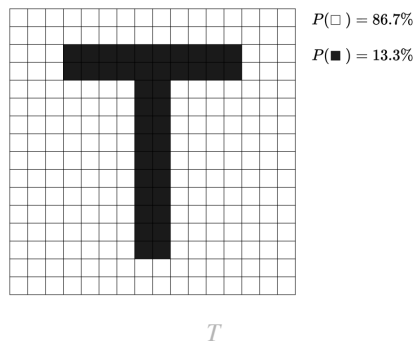
Then the avg length is

$$\begin{aligned} H(X^K) &\leq \mathcal{L}^* < H(X^K) + 1 \\ \frac{H(X^K)}{K} &\leq \frac{\mathcal{L}^*}{K} < \frac{H(X^K)}{K} + \frac{1}{K} \\ \mathcal{L}_S^* &\rightarrow \mathcal{H}(X) \leq H(X) \end{aligned}$$

This improves the **1bit penalty for non dyadic distributions**

However the complexity is exponential with K and the joint entropy is very costly.

This example will cover the different results (calculations not shown) for the following image:



Block size 1: $H(X) = 0.586$, $\mathcal{L}^* = 1$

Block size 2: $H(X_1X_2) = 1.022 \rightarrow H/2 = 0.511$, $\mathcal{L}^* = 1.3 \rightarrow \mathcal{L}_S = 0.65 \text{ bpp}$

Block size 4: $H(\prod X_i) = 1.533 \rightarrow H/4 = 0.383$, $\mathcal{L}^* = 1.733 \rightarrow \mathcal{L}_S = 0.433 \text{ bpp}$

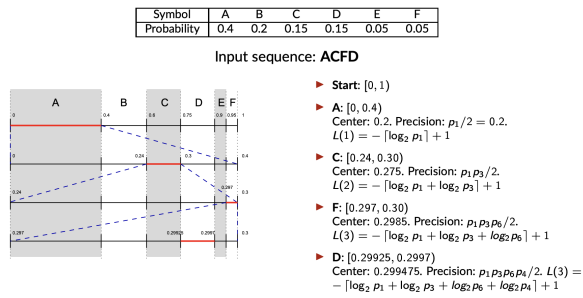
Arithmetic Coding

Arithmetic coding allows to perform block coding or context-based coding with linear complexity. This coder is suboptimal but asymptotically optimal. Since it has linear complexity, it can be easier scaled.

$$\mathcal{L} \leq H(X^K) + 2 \rightarrow \mathcal{L}_s = \frac{\mathcal{L}}{K} \xrightarrow{K \rightarrow \infty} \mathcal{H}(X)$$

The code is built by determining an interval in $[0, 1]$ according to its probability. Needs only 2 sums and 2 multiplications per interval (linear complexity!).

This is done achieved by **encoding a sequence as the center interval** with arbitrary precision $q \in [0, 1]$ a fractional number. The arithmetic code can encode blocks of any size, even the entirety of the message.



Example

Proof of length:

From the above image the length of a message results:

$$L(n) = 1 - \left\lceil \sum \log_2 p_i \right\rceil < 2 - \sum \log_2 p_i$$

The average message length is

$$\bar{L}(n) < \frac{2 - \sum \log_2 p_i}{n}$$

and by taking the expected value the avg length can be found:

$$\mathcal{L} = \mathbb{E}[\bar{L}(n)] < \frac{2 - \sum \mathbb{E}[\log_2 p_i]}{n} = H(X) + \frac{2}{n} \rightarrow \mathcal{L} < H(X)$$

Context Based Coding (TODO)

This approach consists in looking at N_s previous symbols to recognize the context (max $N_c = M^{N_s}$ with M the alphabet) and changes the encoder based on the context. This conditioning reduces the entropy of the source.

Engineering Challenge: with the right context a shorter number of samples manages to better reach the entropy rate.

3.3.2) Recap

Here is a brief recap

Scheme	Length rule	Average-length bound	Complexity	Key idea / limitation
Optimal (theoretical)	$l_i^* = -\log_2 p_i$ (real-valued)	$\mathcal{L}^* = H(X)$	—	Only achievable exactly if p_i is dyadic
Entropy code	$l_i^* = -\lceil \log_2 p_i \rceil$	$H(X) \leq \mathcal{L} \leq H(X) + 1$	low	Simple rounding; up to 1-bit/symbol penalty
Huffman	Greedy bottom-up tree merge	$H(X) \leq \mathcal{L} \leq H(X) + 1$	$O(M \log M)$	Provably optimal among integer-length codes; small-alphabet penalty (e.g. binary source)
Block coding	Code K symbols jointly	$L_S \rightarrow \mathcal{H}(X) \leq H(X)$ as $K \rightarrow \infty$	exponential in K (M^K symbols)	Removes 1-bit penalty asymptotically; explodes in cost
Arithmetic coding	Whole sequence $\rightarrow$ one interval/point	$\mathcal{L}_S < H(X) + \frac{2}{n} \rightarrow H(X)$	linear, $O(n)$	Sub-optimal for finite n , asymptotically optimal; scales where block coding can't
Context-based coding	Condition on N_s previous symbols	lowers effective entropy rate further	depends on $N_c = M^{N_s}$	Exploits memory/correlation; right context model is the engineering challenge

3.4) Other Techniques (TODO)

3.4.1) Exp-Golomb Coding

Universal coding for integer numbers, size (bits) proportional to magnitude

Unsigned Integer

Given int $n \in \mathbb{N}$ the representation consists in

- write $n + 1$ in binary
- use min number of bits: $b = \lfloor \log_2(n + 1) \rfloor + 1$
- place $b - 1$ leading zeroes

Example:

$$\begin{aligned} n = 0 &\rightarrow \begin{cases} n + 1 = 1_{10} \rightarrow 1_2 \\ b = \lfloor \log_2 1 \rfloor + 1 = 1 \rightarrow 1 \\ b - 1 = 0 \end{cases} \\ n = 6 &\rightarrow \begin{cases} n + 1 = 7_{10} = 111_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 3 \rightarrow 00111 \\ b - 1 = 2 \end{cases} \end{aligned}$$

Signed Integer

Given int $n \in \mathbb{Z}$ the representation consists in

- Map $\mathbb{Z} \rightarrow \mathbb{N}$ using $m(n) = \begin{cases} 2n - 1 & n > 0 \\ -2n & n \leq 0 \end{cases}$
- Use Exp-Golomb for unsigned integer

Example:

$$\begin{aligned} n = -3 &\rightarrow m(n) = 6 \rightarrow 00111 \\ n = -6 &\rightarrow m(n) = 12 \rightarrow \begin{cases} n + 1 = 13_{10} = 1101_2 \rightarrow 1_2 \\ b = \lfloor \log_2 13 \rfloor + 1 = 4 \rightarrow 0001011 \\ b - 1 = 3 \end{cases} \end{aligned}$$

3.5) Standards (TODO)

JBIG

JPEG-LS

PNG

3.6) Neural Lossless Coding (TODO)

4) Transform Coding

Until now we have seen the following pipeline:

$$\text{Data} \xrightarrow{\text{lossy}} \text{Quantized data at fixed rate} \xrightarrow{\text{lossless}} \text{Entropy Coded Data}$$

The real pipeline inserts two stages before quantization:

- **Transform coding:** a reversible transform that sparsifies the block and gives each sample position a stationary meaning (e.g. DCT coefficient 0 = mean energy, the rest = increasing frequencies).
- **Block coding:** find the optimal rate per sample, that is, how sensible should be the quantizer for each sample.
- **Quantize:** apply those quantizers, this exploits unequal variances (high variance, many bits in quantizer, low variance only a few bits)
- **Entropy code:** pack the resulting bitstream exploiting the many resulting zeros to approach the entropy.

We start with **block coding**, because it defines *what we want from a transform and why*. But it must be clear from the start that block coding alone is useless on raw natural images (see [Example \(ID RVs \(PCM\)\)](#)), for two distinct reasons:

- **No variable rate:** all pixels have (approximately) equal variance, so we revert to UQ
- **Too many quantizers:** even with unequal variances, one fixed quantizer can serve every block only if each position's variance is stationary across blocks (a constant per-sample meaning, as the transform provides, is one way to guarantee this).

The transform is precisely what supplies both: variance disparity (gain) and cross-block stationarity (one shared quantizer).

Digression: Arithmetic Mean (AM) vs Geometric Mean (GM)

First we must clarify the difference between AM and GM:

$$z_{AM} = \frac{1}{M} \sum_{k=1}^M z_k \quad z_{GM} = \sqrt[M]{\prod_{k=1}^M z_k}$$

We will see that in orthonormal transforms

- AM is unchanged ($\propto \mathcal{L}^2$ norm)
- GM can be reduced (energy concentration)

By **Jensen's inequality**:

$$z_{GM} \leq z_{AM}$$

Proof

Jensens inequality states that:

$$\sum_{k=1}^M \frac{1}{M} f(z_k) \leq f\left(\sum_{k=1}^M \frac{1}{M} z_k\right) = f(z_{AM})$$

with f the log function:

$$\sum \frac{1}{M} \log z_k = \log\left(\prod z_k^{\frac{1}{M}}\right) = \log(z_{GM}) \leq \log(z_{AM})$$

□

4.2) Block Coding

Block coding aims to solve the **resource allocation problem**, that is, find the rate vector $R = [R_1, \dots, R_M]$ that minimizes global distortion on block $X = [X_1, \dots, X_M]$ under the constraint $\sum R_k = R_{tot}$. That is a different quantizer per sample

Recall that an optimal HR quantizer has a distortion for a sample of $D_k = c_k \sigma_k^2 2^{-2R_k}$. The **global distortion** is:

$$\begin{aligned} \mathcal{D} &= \frac{1}{M} \mathbb{E}[\|X_Q(X)\|^2] = \frac{1}{M} \mathbb{E}[(X - Q(X))^T (X - Q(X))] \\ &= \frac{1}{M} \mathbb{E}[\sum_k (X_k - Q(X_k))^2] = \frac{1}{M} \sum_k \mathbb{E}[(X_k - Q(X_k))^2] = \frac{1}{M} \sum_k D_k \\ &= \frac{1}{M} \sum_{k=1}^M c_k \sigma_k^2 2^{-2R_k} \end{aligned}$$

The solution is the **Huang-Schulteiss formula**:

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2} \log_2 \left[\frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2} \right]$$

That is, the rate is a refinement of a uniform resource allocation $\bar{R} = R_{tot}/M$.

The per sample distortion becomes:

$$D_k^* = c_k \sigma_k^2 2^{-2\bar{R} - \log_2(\cdot)} = c_k \sigma_k^2 2^{-2\bar{R}} \cdot \frac{c_{GM} \sigma_{GM}^2}{c_k \sigma_k^2} = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$

And the global distortion only depends on the geometric mean:

$$\mathcal{D}^* = \frac{1}{M} \sum_k D_k^* = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$

Here are two examples:

Example (Gaussian Optimal Rate).

Recall that for a gaussian rv the shape factor is constant and has this relation: $c_{GM} = c_k = c_N$, then clearly:

$$\mathcal{D}^* = c_N \sigma_{GM}^2 2^{-2\bar{R}}$$

Example (ID RVs (PCM)).

When the k rvs are id we have that shape factor and variance are constant, therefore:

$$c_k = c_X = c_{GM} \quad \sigma_k^2 = \sigma_X^2 = \sigma_{GM}^2$$

Then

$$R_k^* = \bar{R} + \frac{1}{2} \log \frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2} = \bar{R} \quad \mathcal{D}^* = c_X \sigma_X^2 2^{-2\bar{R}}$$

the signal is not sparse at all, so block coding does not work

If the ID signals are zero mean gaussian rvs, then we call it PCM, the coding distortion becomes

$$\mathcal{D}_X = c_N \sigma_{GM}^2 2^{-2\bar{R}}$$

PROOF (TODO)

The function to minimize and the constraint are:

$$\mathcal{D} = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \quad \sum_{k=0}^{M-1} R_k \leq R_{tot}$$

By lagrange method we must minimize:

$$J(R, \lambda) = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} + \lambda \left(\sum_{k=0}^{M-1} R_k - R_{tot} \right)$$

The derivatives become:

$$\frac{\partial J}{\partial R_k} = -\frac{2 \ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda = 0 \quad \frac{\partial J}{\partial \lambda} = \sum_{k=0}^{M-1} R_k - R_{tot} = 0$$

from the first we get:

$$R_k = \frac{1}{2} \log_2(c_k \sigma_k^2) + \underbrace{\frac{1}{2} \log_2 \frac{2 \ln 2}{M \lambda}}_{\text{constant: } \lambda'}$$

Plugging it into the second we have:

$$\begin{aligned} R_{tot} &= M \lambda' + \sum \frac{1}{2} \log_2(c_k \sigma_k^2) \rightarrow \lambda' = \frac{R_{tot}}{M} - \frac{1}{2M} \sum \log_2(c_k \sigma_k^2) \\ &= \bar{R} - \frac{1}{2} \log_2 \prod (c_k \sigma_k^2)^{1/M} \\ &= \bar{R} - \frac{1}{2} \log_2(c_{GM} \sigma_{GM}^2) \end{aligned}$$

plug it again in the initial formula:

$$R_k^* = \frac{1}{2} \log_2(c_k \sigma_k^2) + \lambda' = \bar{R} + \frac{1}{2} \log_2 \frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2}$$

□

Practical Implementation of Huang-Schulteiss

HS has a series of limitations:

- Returns negative values
- Returns non-integer values

The **Modified HS algorithm** is very simple:

1. Use HS with all M components
2. If there are negative values, set $R = 0$ of the component and recalculate HS with $M - N$ components
3. When there are only positive values, floor the results
4. Calculate residual rate and allocate eventual residual bits to the results that got rounded the most

Let there be a gaussian input with 4 components and $R_{tot} = 10$:

$$\sigma_1^2 = 1000, \sigma_2^2 = 100, \sigma_3^2 = 50, \sigma_4^2 = 1$$

The GM is ≈ 47.29

We get:

$$R(1) \approx 4.7, R(2) \approx 3.04, R(3) \approx 2.54, R(4) \approx -0.28 \xrightarrow{\text{negative}} 0$$

Recompute HS with 1,2,3:

$GM \approx 171$

We get (also floor):

$$R(1) \approx 4.61 \rightarrow 4, R(2) \approx 2.95 \rightarrow 2, R(3) \approx 2.45 \rightarrow 2, R(4) = 0 \rightarrow 0$$

Since $4 + 2 + 2 + 0 \neq 10$ we can allocate 2 residual bits to components 1 and 2

$$R(1) = 5, R(2) = 3, R(3) = 2, R(4) = 0$$

The total distortion is:

$$D = \sum D_i = \sum \sigma_k^2 2^{-2R_k} = 0.98 + 1.56 + 3.13 + 1 = 6.67$$

With equal rate it was:

$$D = \sum \sigma_k^2 2^{-2 \cdot 2.5} = 35.97$$

The **greedy algorithm** returns the same allocation but is faster:

- Initialization. $R_k = 0 \forall k \in \{0, \dots, M-1\}$ $D_k = \sigma_k^2 \forall k \in \{0, \dots, M-1\}$
 - While $\sum R_k \leq R_{tot}$
 - $l = \arg \max_k D_k$
 - $R_l \leftarrow R_l + 1$
 - $D_l \leftarrow D_l/4$ (since $(D(R+1) = \sigma_k^2 2^{-2(R+1)} = \frac{1}{4} \sigma_k^2 2^{-2R} = D(R)/4)$)
- This takes R_{tot} iterations

Consider the same data as before:

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
0	-	0	0	0	0	1000.00	100.00	50.00	1.00
1	1	1	0	0	0	250.00	100.00	50.00	1.00
2	1	2	0	0	0	62.50	100.00	50.00	1.00
3	2	2	1	0	0	62.50	25.00	50.00	1.00
4	1	3	1	0	0	15.63	25.00	50.00	1.00
5	3	3	1	1	0	15.63	25.00	12.50	1.00

Iterations 1-5

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
6	2	3	2	1	0	15.63	6.25	12.50	1.00
7	1	4	2	1	0	3.91	6.25	12.50	1.00
8	3	4	2	2	0	3.91	6.25	3.13	1.00
9	2	4	3	2	0	3.91	1.56	3.13	1.00
10	1	5	3	2	0	0.98	1.56	3.13	1.00

Iterations 6-10

4.3) Transform Coding

A non-sparse signal codes badly. This chapter looks for a **transform that sparsifies** the signal (few large coefficients, many small ones).

We want a transform with the following properties:

- **Invertible:** $Y = T(X) \iff X = T^{-1}(Y)$
- **Input in $\mathbb{R}^M$** (e.g. an image lives in $\mathbb{R}^2$)
- **Non-sparse input $\rightarrow$ sparse output, at the same quantization error**

We restrict to a **linear** transform $Y = \mathcal{T}X$, with $\mathcal{T}$ an invertible matrix:

- inverse exists by definition;
- it acts as a **change of basis** (the columns of $\mathcal{T}^{-1}$ are the signals we reconstruct with);
- if $\mathcal{T}$ is **orthonormal**, quantization error is preserved (energy is conserved);
- we then seek the $\mathcal{T}$ that **minimizes the GM** of Y (maximal energy concentration).

Paradigm:

$$x \rightarrow y = \mathcal{T}x \rightarrow \hat{y} = Q(y) \rightarrow \hat{x} = \mathcal{T}^{-1}\hat{y}$$

A coding quality is determined by the coding gain:

$$G_{\mathcal{T}} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2}$$

Orthogonal Transform

For OT we have:

- Inverse is immediate: $\mathcal{T}^{-1} = \mathcal{T}^T$
- They are isometries (keep $\mathcal{L}^2$ norm): $\|Y\|^2 = \|\mathcal{T}X\|^2 = \|X\|^2$

Since it is an isometry, the distortion is the same:

$$MD_Y = \mathbb{E}[\|Y - \hat{Y}\|^2] = \mathbb{E}[\|\mathcal{T}X - \mathcal{T}\hat{X}\|^2] = \mathbb{E}[\|\mathcal{T}(X - \hat{X})\|^2] = \mathbb{E}[\|X - \hat{X}\|^2] = MD_X$$

And also the AM (avg of variances):

$$\sigma_{AM,Y}^2 = \frac{1}{M} \sum \mathbb{E}[Y_k^2] = \frac{1}{M} \mathbb{E}[\sum Y_k^2] = \frac{1}{M} \mathbb{E}[\|Y\|^2] = \frac{1}{M} \mathbb{E}[\|X\|^2] = \sigma_{AM,X}^2$$

Where logically, the quantizers used are the ones that block code optimized for Y , so we can call this distortion the **transform distortion** $D_{\mathcal{T}}$ ($D_{\mathcal{T}} = D_X = D_Y$), while the **uniform rate (PCM)** distortion is D_{PCM} .

We define the **coding gain** as the ratio:

$$G_{\mathcal{T}} = \frac{D_{PCM}}{D_{\mathcal{T}}} = \frac{c \sigma_{AM}^2 2^{-2\bar{R}}}{c \sigma_{GM,Y}^2 2^{-2\bar{R}}} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} \geq 1$$

PCM distortion (input is id $\rightarrow$ equal variances $\rightarrow \sigma_{GM}^2 = \sigma_{AM}^2 \rightarrow$ trivial allocation, same rate $\bar{R}$):

$$D_{PCM} = c_X \sigma_X^2 = c \sigma_{AM}^2 2^{-2\bar{R}}$$

Transform distortion (isometry fixes σ_{AM}^2 ; equal shape factors on avg):

$$D_{\mathcal{T}} = c \sigma_{GM,Y}^2 2^{-2\bar{R}}$$

Optimal Transform (KLT)

An optimal transform is the the transform that, given a random vector X with known statistical properties, has a mathematically optimal guarantee to decorrelate the components and maximizes energy compaction.

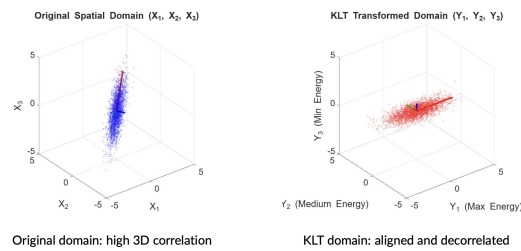
The answer is the **Karhunen-Loève Transform (KLT)**

The rows of this matrix are the eigenvectors of the correlation matrix $R_X = \mathbb{E}[XX^T]$. This allows the transform to be oriented along the maximum variance direction

Intuition: In the Y domain, knowing the value of Y_1 provides zero linear information about Y_2 . The redundancy present in the "diagonal" alignment is removed

Properties:

- Orthogonality: $T_{KLT}^{-1} = T_{KLT}^T$
- Decorrelating transform: $\mathbb{E}[Y_i Y_j] = \lambda_i \delta_{ij}$
- Best energy concentration (sparsity): $\sum \mathbb{E}[Y_i^2] \geq \sum \mathbb{E}[(TX)^2]$ where T is any other transform
- Optimal for gaussian RV: $\sigma_{GM,Y}^2 \leq \sigma_{GM,TX}^2$



Example

Most coefficients will have near zero variance, so these can be discarded or coarsely quantized since they don't impact the MSE much.

It maximizes the coding gain as Gaussians will be i.i.d.:

$$G_T = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} = \frac{\frac{1}{M} \sum \sigma_i^2}{(\prod \sigma_i^2)^{1/M}}$$

However:

- Requires $O(n^3)$ to find eigenvectors, $O(n^2)$ for the multiplication
- Since data dependent, it must be sent to the decoder as metadata
- Model is often not stationary!

Frequency Transforms (DFT, DCT)

For a Markov process with correlation $\rho \rightarrow 1$ frequency transforms offer near optimal performance (with fixed basis functions and FFT algorithms $O(n \log n)$).

Discrete Fourier Transform (DFT)

The 1D case is the following:

$$y[k] = \frac{1}{\sqrt{M}} \sum_{n=1}^M x[n] e^{-j \frac{2\pi}{M} kn}$$

Or in matrix form:

$$\mathcal{T}_{DFT} = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & W_M & W_M^2 & \dots & W_M^{M-1} \\ 1 & W_M^2 & W_M^4 & \dots & W_M^{2(M-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_M^{M-1} & W_M^{2(M-1)} & \dots & W_M^{(M-1)(M-1)} \end{bmatrix}$$

Where $W_M = e^{-j2\pi/M}$ is the M-th primitive root of unity
Each row is therefore the conjugate of a basis vector.
the total energy is preserved: $\|y\|^2 = \|x\|^2$

In 2D, the DFT is a separable transform:

$$Y = \mathcal{T} X \mathcal{T}^T$$

this computes the rows and then the columns (horizontal and vertical frequency analysis)
This transform decomposes the image into a weighted sum of N^2 orthogonal basis patterns:

$$B_{k,l}(n, m) = \frac{1}{N} e^{j\frac{2\pi}{N}(kn+lm)}$$

each coefficient $Y[k, l]$ represents the frequency component of horizontal and vertical frequency combination
Most energy is concentrated in the low frequencies.

This is still not ideal since the DFT supposes a periodic signal. We compress finite signals and therefore on the image edges we have spectral leakage (low frequencies leak into high ones) making the signal less sparse

DISCRETE COSINE TRANSFORM (DCT)

A general approach is used **Discrete Cosine Transform (DCT)**

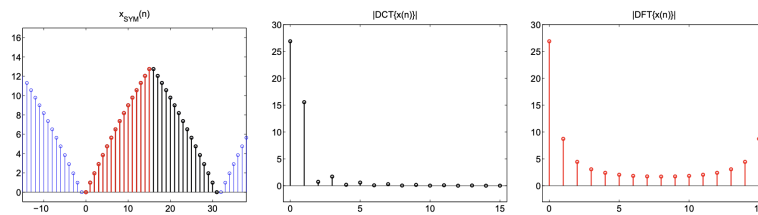
- Create mirrored-periodic version of the signal
- Compute DFT of signal+mirror
- Apply frequency domain modulation to obtain symmetrical and real valued coefficients
- Keep only M coefficients (og signal length)

Which can be rewritten in a single transform where each entry follows the form:

$$(\mathcal{T}_{DCT})_{k,n} = \begin{cases} \frac{1}{\sqrt{N}} & k = 0 \\ \sqrt{\frac{2}{N}} \cos\left(\frac{(2n+1)k\pi}{2N}\right) & k > 0 \end{cases}$$

DFT is not used since DFT has high frequency components near the signal edges. The DCT is a way to mirror the signal before the periodicity. It has only positive frequencies

Applying the DCT to a signal (a sequence of M real numbers) produces M real coefficients and has a better sparsification property than DFT thanks to the symmetric periodization.



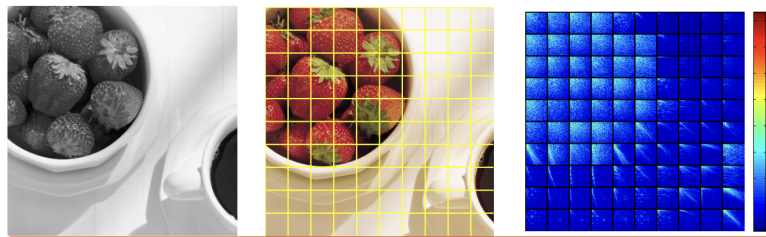
Example With Mirroring

The DCT is also separable:

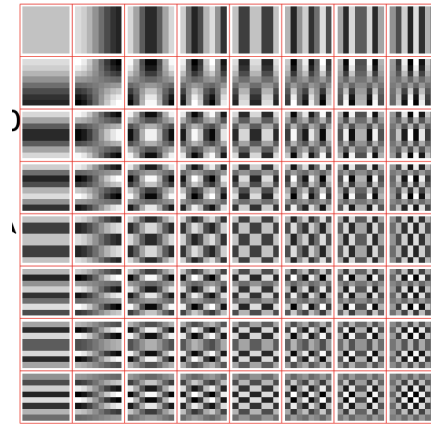
$$Y = \mathcal{T}_{DCT} X \mathcal{T}_{DCT}^T$$

A large-size, non-stationary image is more conveniently represented by dividing it into small blocks.

Example: 8×8 block-based DCT. Each 8×8 block of pixels from the image is projected onto the 64 basis vectors:
The corresponding scalar product is the DCT coefficient telling how much the block is similar to the basis vector



Block DCT



8x8 Basis

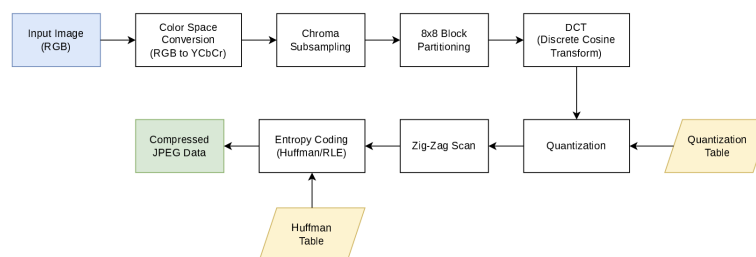
The sparsification allows to give higher bits to many small valued coefficients and lower bits to less frequent bigger values. How are the quantization coefficients computed?

- HS formula (practical implementation)
- Fixed steps (JPEG)

Large blocks: stationarity
Small blocks: correlation

4.4) JPEG standard

JPEG is an image compression standard defined in 1991 that defines **only the decoder** for interoperability and implementation competition



JPEG Scheme

This is the encoder:

Step 0; Preprocessing: includes 8×8 block creation and centering (-128 on each sample).

Step 1; DCT: 2D DCT is applied on 8×8 blocks of centered samples (find y_{ij})

Step 2; Quantization: uses a **mid tread** quantizer. The quantization table q_{ij} is not defined by the standard, therefore it must be encoded. Notice that q is the step size. A bigger q means fewer bits and therefore less quality (as an idea think of it like this $q = 255/2^R$)

$$\hat{c}_{ij} = \text{round}\left(\frac{y_{ij}}{q_{ij}}\right)$$

Step 2.5; Quality Factor: The **quality factor** $Q \in [1, 100]$ that controls the scaling factor (used during quantization):

$$S_F = \begin{cases} \frac{5000}{Q} & 1 \leq Q \leq 50 \\ 200 - 2Q & 50 < Q \leq 99 \\ 1 & Q = 100 \end{cases} \rightarrow q \leftarrow \frac{S_F}{100} q$$

Step 3; Zig-Zag Scan + Entropy coding: see later

Now the decoder:

Step 0; Entropy Decode: from zigzag + entropy decode it to find quantized blocks.

Step 1; De-Quantization: multiply quantized coefficients by q .

$$\hat{y}_{ij} = \hat{c}_{ij} \cdot q_{ij}$$

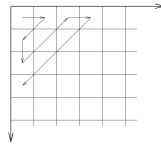
Step 2; Inverse DCT: apply the IDCT on each dequantized block and reconstruct the signal by uncentering (+128 on each sample)

Zig Zag Scan:

A zig-zag scan is performed on the quantized values in order to encode them in a single string, where

- the first value is the Difference of the DC component of this block and the previous block
- the next values are a pair of numbers representing (# of zeroes in scan, value of first non zero)
- final EOB special symbol is added to end the string it is (0, 0)

$$DC_n - DC_{n-1} \quad | \quad (\# \text{ of zeroes, non zero coeff value}) \quad | \quad \dots \quad | \quad \text{EOB (0, 0)}$$



Zig-Zag scan

-2	17	3	1	0	0	0	0
3	4	0	-1	0	0	0	0
0	0	-1	0	0	0	0	0
0	0	-1	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

-2-DC _{n-1}	0,17	0,3	1,4	0,3	0,1	5,-1	0,-1	4,-1	5,-1	EOB
----------------------	------	-----	-----	-----	-----	------	------	------	------	-----

Example

ENTROPY CODING OF DC COEFFICIENTS

JPEG encodes the DC difference with a pseudo-huffman code.

There are $k \in \{0, \dots, 11\}$ categories, each of them holding 2^k values using 2's complement. Each category is assigned a codeword (see table).

Call DC_p the DC difference, the category is chosen $k = \lceil \log_2(|DC_p| + 1) \rceil$

To the category the binary value of DC_p is added as a suffix. If it is negative each value is complemented.

Category	Code
0	00
1	010
2	011
3	100
4	101
5	110
6	1110
7	11110
8	111110
9	1111110
10	11111110
11	111111110

Category Code

For example suppose $DC_P = -5_{10} = 101_2$ that must be complemented to 010. The category is $k = 3 \rightarrow 100$ and thus the codeword is 100 010.

DC values have a range $\in [-1024, 1060]$ a difference of two DC signals is $\in [-2040, 2040]$ and thus with cardinality 4081. Since there are 11 categories we have $\sum_{k=0}^{11} 2^k = 2^{12} = 1096 > 4081$ values.

ENTROPY CODING OF AC COEFFICIENTS

Recall that these coefficients are previously encoded as (a, b) . These values are used to find the pseudo huffman entropy encoded codeword which is prefix+:

$$[(a, k) \text{ in RC table prefix} \mid b \text{ DC pseudo huffman}]$$

That is:

- First compute $k = \lceil \log_2(|b| + 1) \rceil$ and b in binary
- Then find the prefix code corresponding to (a, k) (table)
- Put prefix + b together

Example: Encode (3,16):

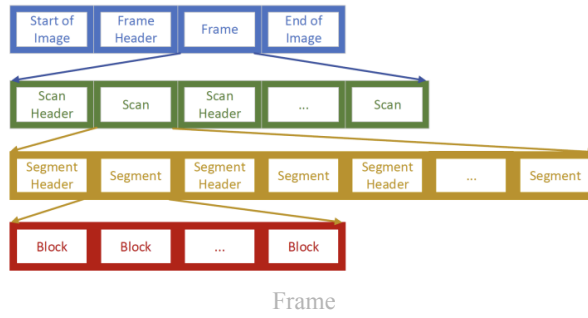
- $k = \lceil \log_2 17 \rceil = 5$ and $b = 10000_2$
- In table see that $(3, 5) = 111111110010000$
The codeword is: 111111110010000 10000

Two custom codewords are described:

- End Of Block (EOB) = $(0, 0) \rightarrow 1010$
- Zero Run (ZR) = $(15, 0) \rightarrow 11111111001$

Frame Building

The standard frame follows this logic:



- Frame header contains static info (size, color space, digitalization format)
- Image is stored in a frame as various scans
- Scan header contains quantization table (luminance and chrominance)
- A single scan contains various segments, each segment is a concatenation of blocks
- Segment header contains huffman tables

JFIF (JPEG File Interchange Format) is the standard format for metadata in JPEG files

4.5) Chapter Recap

Block coding (resource allocation problem)

Minimize $\mathcal{D} = \frac{1}{M} \sum_k c_k \sigma_k^2 2^{-2R_k}$ subject to $\sum_k R_k = R_{tot}$.

- Optimal rate (Huang–Schultheiss):

$$R_k^* = \bar{R} + \frac{1}{2} \log_2 \frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2}, \quad \bar{R} = \frac{R_{tot}}{M}$$

- Optimum **equalizes** per-sample distortion, so the global distortion depends only on the GM:

$$\mathcal{D}^* = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$

AM vs GM. $z_{GM} \leq z_{AM}$ (Jensen, log concave), equality $\iff$ all z_k equal.

Coding gain. Orthonormal transform conserves energy (σ_{AM}^2 fixed by isometry, $D_X = D_Y$) but lowers σ_{GM}^2 (energy concentration):

$$G_{\mathcal{T}} = \frac{D_{PCM}}{D_{\mathcal{T}}} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} \geq 1, \quad = 1 \iff \text{all } \sigma_k^2 \text{ equal}$$

$$D_{PCM} = c \sigma_{AM}^2 2^{-2\bar{R}}, \quad D_{\mathcal{T}} = c \sigma_{GM,Y}^2 2^{-2\bar{R}}$$

$\Rightarrow$ block coding helps **only with variance disparity**; the transform exists to create it.

KLT. Rows = eigenvectors of $R_X = \mathbb{E}[XX^T]$. Decorrelates ($\mathbb{E}[Y_i Y_j] = \lambda_i \delta_{ij}$); optimal energy compaction (top- p partial energy maximal $\forall p$); minimizes $\sigma_{GM,Y}^2 \Rightarrow$ maximizes $G_{\mathcal{T}}$. For Gaussian X : decorrelation $\Rightarrow$ **independence** $\Rightarrow$ rate–distortion optimal. Cost: $O(n^3)$, data-dependent (basis must be transmitted), assumes stationarity.

DCT. Fixed basis, $O(n \log n)$, no basis to transmit; near-optimal for Markov sources with $\rho \rightarrow 1$. Symmetric periodization avoids DFT edge leakage $\rightarrow$ real, sparser coefficients.

Modified Huang–Schultheiss (fixes negative / non-integer rates)

1. Run HS on all M components.
2. Any $R_k < 0 \rightarrow$ set $R_k = 0$, drop that component, re-run HS on the rest.
3. Repeat until all rates ≥ 0 ; then **floor** them.

4. Distribute leftover bits ($R_{tot} - \sum \lfloor R_k \rfloor$) to the components rounded down the most.

Greedy bit allocation (same result, $O(R_{tot})$)

- Init: $R_k = 0, D_k = \sigma_k^2 \forall k$.
- Repeat R_{tot} times:
 - $l = \arg \max_k D_k$
 - $R_l += 1$, then $D_l /= 4$ (since one extra bit quarters distortion).

KLT construction

1. Estimate $R_X = \mathbb{E}[XX^T]$.
2. Eigendecompose: $R_X = U\Lambda U^T$.
3. Transform $Y = U^T X$ (rows of $\mathcal{T}_{KLT} = U^T$ are the eigenvectors).
4. Coefficient Y_i has variance λ_i ; keep / finely quantize large λ_i , discard small ones.

JPEG encoder

0. **Preprocess:** 8×8 blocks, level-shift -128 .

1. **DCT:** 2-D DCT per block $\rightarrow y_{ij}$.
2. **Quantize:** build table (base $\times$ quality Q , clamp $\max(1, \cdot)$); mid-tread round $\ell_{ij} = \text{round}(y_{ij}/q_{ij}) \rightarrow$ small coeffs become 0.
3. **Zig-zag + RLE:** DC differential ($DC_n - DC_{n-1}$); AC as (run-of-zeros, value); EOB (0, 0), ZRL (15, 0).
4. **Entropy code:** pseudo-Huffman (category $k = \lceil \log_2(|v| + 1) \rceil + \text{magnitude bits}$) $\rightarrow$ bitstream. Quant + Huffman tables go in the header.

JPEG decoder (reverse)

0. **Entropy decode** bitstream $\rightarrow$ levels ℓ_{ij} .

1. **De-quantize:** $\hat{y}_{ij} = \ell_{ij} q_{ij}$.
2. **IDCT** per block.
3. **Uncenter** $+128$, clamp to $[0, 255]$.

Block coding defines *what a good transform must do* (maximize $\sigma_{AM}^2 / \sigma_{GM}^2$) $\rightarrow$ KLT does it **optimally** $\rightarrow$ DCT does it **cheaply** $\rightarrow$ JPEG **packages** it. Transform creates redundancy as variance disparity, allocation spends bits on it, entropy coding harvests the resulting zeros.

Wavelet Transform